SESSION 1: INTRODUCTION TO R AND RSTUDIO

Foundations for Data Science Workflow

LUMUMBA W.V.

Center for Data Analytics and Modeling
Chuka University

March 2026

Resource: https://cdam.chuka.ac.ke

Learning Objectives

By the end of this session, you will be able to:

1. Understand R's role in modern data science
2. Navigate RStudio interface efficiently
3. Work with basic R data types and structures
4. Perform basic operations and functions
5. Import and inspect data files
6. Apply reproducible workflow practices
7. Access help and documentation effectively
8. Set up your environment for data science work

Session Outline

1. Why R for Data Science
2. R History and Ecosystem
3. RStudio Interface Overview
4. Console and Script Workflow
5. Basic Data Types
6. Vectors and Operations
7. Matrices and Arrays
8. Lists and Data Frames
9. Factors for Categorical Data
10. Basic Functions and Operators
11. Working Directory Management
12. Importing CSV Data
13. Inspecting Data Structure
18. Reproducible Workflow Setup
19. R Packages and Libraries
20. Getting Help in R
21. Best Practices for Beginners
22. Common Beginner Mistakes
23. Debugging Basics
24. Code Organization Tips
25. Version Control Introduction
26. R Markdown Overview
27. Project Structure Guidelines
28. Performance Considerations
29. Resources for Learning

Why R for Data Science

R is designed for statistical computing and graphics:

- Open source and free for all platforms
- Comprehensive statistical and machine learning libraries
- Excellent data visualization capabilities (ggplot2)
- Strong community and academic support
- Reproducible research tools (R Markdown, knitr)
- Integration with databases, web APIs, and big data tools

Industry Adoption:

- Finance, healthcare, tech, government, academia
- Used by Google, Facebook, Microsoft, Pfizer, and more

Key: R combines statistical rigor with practical data science workflow

R History and Ecosystem

Brief History:

- 1993: R created by Ross Ihaka and Robert Gentleman
- Based on S language from Bell Labs (1970s)
- 2000: R Core Team formed for development
- Today: Over 19000 packages on CRAN

Key Components:

- **Base R**: Core language and functions
- **CRAN**: Comprehensive R Archive Network
- **RStudio/Posit**: Integrated development environment
- **Tidyverse**: Modern data science package collection
- **Bioconductor**: Genomics and bioinformatics tools

Benefit: Mature ecosystem with solutions for nearly every data task

RStudio Interface Overview

Four Main Panes:

1. **Source Editor** (top-left): Write and edit scripts
2. **Console** (bottom-left): Execute commands interactively
3. **Environment/History** (top-right): View objects and command history
4. **Files/Plots/Packages/Help** (bottom-right): Manage files, view outputs

Key Features for Data Scientists:

- Syntax highlighting and code completion
- Integrated help and documentation
- Plot viewer with export options
- Package manager and version control integration
- R Markdown support for reproducible reports

Tip: Learn keyboard shortcuts to speed up your workflow

Console and Script Workflow

Two Ways to Run Code:

```
# Console: Direct execution (good for testing)
> 2 + 2
[1] 4
> x <- c(1, 2, 3, 4, 5)
> mean(x)
[1] 3

# Script: Save and run code (good for reproducibility)
# Save as analysis.R, then run with:
# Source > Run All or Ctrl+Shift+Enter

# Best practice: Write in scripts, test in console
# Always comment your code
# Use meaningful variable names
```

Rule: Scripts are for sharing and reproducing; console is for exploration

Basic Data Types

R has six atomic data types:

```r
# Numeric (double precision)
x <- 3.14
class(x)  # "numeric"

# Integer (add L suffix)
y <- 10L
class(y)  # "integer"

# Character (text strings)
name <- "Data Scientist"
class(name)  # "character"

# Logical (TRUE/FALSE)
is_valid <- TRUE
class(is_valid)  # "logical"

# Complex (rarely used in data science)
z <- 1 + 2i
```

Vectors: The Foundation

Vectors are homogeneous sequences (same data type):

```r
# Create vectors
nums <- c(1, 2, 3, 4, 5)
chars <- c("A", "B", "C")
logs <- c(TRUE, FALSE, TRUE)

# Vectorized operations (apply to all elements)
nums * 2           # [1] 2 4 6 8 10
nums + c(1, 2)     # Recycling: [1] 2 4 4 6 6

# Common vector functions
length(nums)       # 5
sum(nums)          # 15
mean(nums)         # 3
max(nums)          # 5
nums[3]            # Access third element: 3
nums[c(1, 3, 5)]   # Access multiple: 1 3 5
```

Key: Vectorization makes R code efficient and concise

Vector Operations and Indexing

```r
# Logical indexing
ages <- c(25, 30, 35, 40, 45)
ages > 30                # [1] FALSE FALSE TRUE TRUE TRUE
ages[ages > 30]          # [1] 35 40 45

# Named vectors
scores <- c(Math = 85, Science = 92, English = 78)
scores["Science"]        # 92
names(scores)            # "Math" "Science" "English"

# Sequence generation
1:10                     # 1 2 3 ... 10
seq(0, 1, by = 0.1)      # 0.0 0.1 0.2 ... 1.0
rep("A", times = 3)      # "A" "A" "A"

# Combining vectors
c(1:3, 10:12)            # 1 2 3 10 11 12
```

Tip: Use logical indexing for efficient data filtering

Matrices and Arrays

Matrices: 2D homogeneous structures

```r
# Create matrix
mat <- matrix(1:12, nrow = 3, ncol = 4)
mat

# Access elements
mat[2, 3]         # Row 2, Col 3: 8
mat[2, ]          # Entire row 2
mat[, 3]          # Entire column 3

# Matrix operations
t(mat)            # Transpose

# Arrays: multi-dimensional (3D+)
arr <- array(1:24, dim = c(3, 4, 2))
```

Note: Data frames are more common than matrices for data science

Lists: Heterogeneous Collections

Lists can contain different data types and structures:

```r
# Create list
my_list <- list(
  id = 1:5,
  name = c("Alice", "Bob", "Charlie"),
  scores = c(85, 92, 78),
  passed = c(TRUE, TRUE, FALSE)
)

# Access list elements
my_list$name            # By name
my_list[[2]]            # By position

# Add/modify elements
my_list$grade <- c("A", "A", "B")

# Convert list to data frame (if structure allows)
as.data.frame(my_list[1:3])
```

Data Frames: Tabular Data

Data frames are the primary structure for data science:

```r
# Create data frame
df <- data.frame(
  id = 1:4,
  name = c("Alice", "Bob", "Charlie", "Diana"),
  age = c(25, 30, 35, 28),
  income = c(50000, 65000, 72000, 58000),
  stringsAsFactors = FALSE
)

# Inspect data frame
str(df)         # Structure
head(df)        # First 6 rows
summary(df)     # Summary statistics
dim(df)         # Dimensions: 4 rows, 4 columns

# Access columns
df$name                     # By name
df[["income"]]              # By name
```

Data Frame Operations

```r
# Filter rows
df[df$age > 28, ]           # Rows where age > 28

# Add new column
df$income_per_year <- df$income / df$age

# Modify existing column
df$age <- df$age + 1        # Add 1 year to all ages

# Remove column
df$income_per_year <- NULL

# Sort data frame
df[order(df$income), ]      # By income ascending

# Merge data frames
df2 <- data.frame(id = c(1, 2), city = c("Nairobi", "Mombasa"))
merge(df, df2, by = "id")  # Join by common column
```

Note: Tidyverse verbs (filter, mutate, arrange) are more intuitive

Factors for Categorical Data

Factors represent categorical variables with fixed levels:

```r
# Create factor
education <- factor(c("High School", "Bachelor", "Master",
                      "Bachelor", "High School"),
                    levels = c("High School", "Bachelor", "Master"),
                    ordered = TRUE)

# Check factor properties
class(education)        # "factor"
levels(education)       # Level names
nlevels(education)      # Number of levels: 3

# Factor operations
table(education)        # Frequency table
summary(education)      # Summary by level

# Convert between types
as.character(education) # Factor to character
```

Basic Functions and Operators

```r
# Arithmetic operators
5 + 3       # Addition: 8
10 - 4      # Subtraction: 6
6 * 7       # Multiplication: 42
15 / 3      # Division: 5
2^3         # Exponent: 8
10 %% 3     # Modulo: 1

# Comparison operators
5 > 3       # TRUE
5 == 5      # TRUE
5 != 3      # TRUE (not equal)
5 >= 5      # TRUE

# Logical operators
TRUE & FALSE    # AND: FALSE
TRUE | FALSE    # OR: TRUE
!TRUE           # NOT: FALSE
```

Working Directory Management

The working directory is where R looks for files:

```r
# Check current working directory
getwd()

# Set working directory (use forward slashes)
setwd("/home/user/projects/data_science")

# Better: Use R Projects for automatic directory management
# File > New Project > New Directory > New Project

# List files in working directory
list.files()
list.files(pattern = ".csv$")  # Only CSV files

# Create directory
dir.create("results")

# Check if file exists
file.exists("data/GSSsubset.csv")
```

Importing CSV Data

```r
# Base R method
gss_base <- read.csv("GSSsubset.csv",
                     stringsAsFactors = FALSE,
                     na.strings = c("", "NA"))

# Tidyverse method (recommended)
library(readr)
gss <- read_csv("GSSsubset.csv")

# View imported data
head(gss)
glimpse(gss)  # Tidyverse: better than str() for wide data
```

Tip: $\text{read}_c sv() is faster and more consistent than read.csv()$

Inspecting Data Structure

Essential functions for understanding your data:

```r
# Basic inspection
dim(gss)            # Dimensions: rows, columns
nrow(gss)           # Number of rows
ncol(gss)           # Number of columns
names(gss)          # Column names

# Detailed structure
str(gss)            # Base R: compact structure display
glimpse(gss)        # Tidyverse: transposed, easier to read

# View data
head(gss, 10)       # First 10 rows
tail(gss, 5)        # Last 5 rows

# Summary statistics
summary(gss)        # Base R summary for all columns

# Check data types
```

Basic Data Exploration

```r
# Univariate exploration
table(gss$sex)                        # Frequency table
prop.table(table(gss$sex))            # Proportions
summary(gss$income)                   # Numeric summary
quantile(gss$age, probs = c(0.25, 0.5, 0.75))

# Simple visualizations
hist(gss$age, breaks = 20, main = "Age Distribution")
barplot(table(gss$degree), main = "Education Levels")
plot(gss$age, gss$income, main = "Age vs Income")

# Bivariate exploration
table(gss$sex, gss$degree)            # Contingency table
aggregate(income ~ sex, data = gss, mean)
tapply(gss$income, gss$sex, mean)

# Check for unusual values
range(gss$age)                        # Min and max age
unique(gss$marital)                   # Unique marital statuses
```

Handling Missing Values

Missing values are represented as NA in R:

```r
# Identify missing values
is.na(gss$income)                    # Logical vector
sum(is.na(gss$income))               # Count missing in income
colSums(is.na(gss))                  # Missing per column
sum(complete.cases(gss))             # Complete rows

# Filter out missing values
gss_complete <- gss[complete.cases(gss), ]
gss_income_ok <- gss[!is.na(gss$income), ]

# Functions with missing value handling
mean(gss$income, na.rm = TRUE)     # Remove NA before calculation
sum(gss$income, na.rm = TRUE)

# Replace missing values (use cautiously)
gss$income[is.na(gss$income)] <- mean(gss$income, na.rm = TRUE)
```

Caution: Never remove missing data without understanding the mechanism

Simple Data Transformations

```r
# Create new variables
gss$income_k <- gss$income / 1000  # Income in thousands

# Recode variables
gss$age_group <- cut(gss$age,
                     breaks = c(18, 30, 45, 60, 100),
                     labels = c("18-29", "30-44", "45-59", "60+"))

# Transform numeric variables
gss$log_income <- log1p(gss$income)  # log(income + 1)

# Standardize variables
gss$age_z <- scale(gss$age)  # Z-scores: mean=0, sd=1

# Convert data types
gss$sex <- factor(gss$sex)
gss$degree <- factor(gss$degree)

# Conditional transformations
```

Introduction to Tidyverse

Tidyverse: Modern toolkit for data science in R

```r
# Install and load
# install.packages("tidyverse")
library(tidyverse)

# Core packages included:
# - ggplot2: Visualization
# - dplyr: Data manipulation
# - tidyr: Data tidying
# - readr: Data import
# - purrr: Functional programming
# - tibble: Modern data frames
# - stringr: String operations
# - forcats: Factor handling

# Example: Tidyverse vs Base R
# Base R
subset(gss, age > 30 & income > 40000)
```

Reproducible Workflow Setup

Key practices for reproducible data science:

```r
# 1. Use R Projects (File > New Project)
# - Automatic working directory management
# - Keeps related files together

# 2. Script organization
# - Use descriptive file names: 01_import.R, 02_clean.R
# - Add comments and section headers

# 3. Set seed for reproducibility
set.seed(123)  # For any random operations

# 4. Document package versions
sessionInfo()  # Run at end of analysis

# 5. Use relative paths
data <- read_csv("data/GSSsubset.csv")

# 6. Version control with Git
```

R Packages and Libraries

```r
# Install packages (once)
install.packages("dplyr")  # From CRAN

# Load packages (each session)
library(dplyr)    # Load package
require(dplyr)    # Alternative (returns TRUE/FALSE)

# Check installed packages
installed.packages()[, "Package"]  # All installed

# Update packages
update.packages()  # Update all

# View package documentation
?dplyr::filter    # Help for specific function
```

Tip: Load only the packages you need to avoid namespace conflicts

Getting Help in R

Multiple ways to find help:

```r
# Built-in help system
?mean                # Help for mean() function
??"linear model"     # Search help pages for topic
help.start()         # Open help browser

# Search online resources
# - Stack Overflow: https://stackoverflow.com/questions/tagged/r
# - RStudio Community: https://community.rstudio.com

# Package-specific help
package?dplyr        # Help for entire package

# Debugging help
traceback()          # Show error call stack
```

Pro Tip: Read the Examples section first - it is often the fastest way to learn

Best Practices for Beginners

Code Quality:

- Use meaningful variable names (income$_usdnotx1$)
- Add comments for complex logic, not obvious steps
- Keep lines under 80 characters for readability
- Use consistent indentation and spacing

Workflow Habits:

- Save scripts frequently with descriptive names
- Test code in small chunks before running full script
- Keep raw data separate from processed data
- Document assumptions and decisions in code comments

Learning Strategy:

- Start with small, achievable tasks
- Modify working examples rather than writing from scratch

Common Beginner Mistakes

```r
# Mistake 1: Using = instead of == for comparison
if (x = 5) { }       # Wrong: assigns 5 to x
if (x == 5) { }      # Correct: checks if x equals 5

# Mistake 2: Not saving function output
mean(x)              # Prints result but does not save
result <- mean(x)    # Saves to variable for later use

# Mistake 3: Ignoring missing values
mean(x)              # Returns NA if x has missing values
mean(x, na.rm = TRUE)  # Removes NA first

# Mistake 4: Overwriting built-in functions
mean <- 5            # Bad: overwrites mean() function
my_mean <- 5         # Good: use descriptive names

# Mistake 5: Hard-coding file paths
read.csv("C:/Users/Name/...")  # Will not work on other computers
read.csv("data/file.csv")      # Relative path: portable
```

Debugging Basics

When code does not work:

```r
# 1. Read the error message carefully
# Error in mean(x): argument "x" is missing, with no default

# 2. Check object exists and has expected structure
exists("x")            # TRUE/FALSE
str(x)                 # Structure of x
class(x)               # Data type of x

# 3. Test with simple example
mean(c(1, 2, 3))       # Should return 2

# 4. Use print() or cat() to inspect values
print(x)
cat("Value:", x, "\n")

# 5. Check for missing values
sum(is.na(x))

```

Code Organization Tips

Structure your scripts for clarity:

```r
# ==== SECTION HEADERS ====
# Use multiple # or #### for sections
#### Data Import ####
#### Data Cleaning ####
#### Analysis ####

# ==== COMMENTING ====
# Comment WHY, not just WHAT
# Bad: x <- x + 1  # Add 1 to x
# Good: x <- x + 1  # Adjust for inflation

# ==== NAMING CONVENTIONS ====
# Use snake_case for variables and functions
income_usd <- 50000
calculate_mean <- function(x) { mean(x) }

# ==== SPACING ====
# Spaces around operators
```

Version Control Introduction

Git and GitHub for R users:

```
# Why use version control?
# - Track changes to code over time
# - Revert to previous versions if needed
# - Collaborate without overwriting work

# Basic Git workflow in RStudio:
# 1. Initialize repository: Tools > Version Control > Initialize
# 2. Stage changes: Git pane > check files to commit
# 3. Commit: Add message describing changes
# 4. Push: Send to remote repository (GitHub)

# .gitignore file: Exclude files from version control
# Common exclusions: .Rhistory, .RData, *.Rproj.user
```

Start Simple: Commit daily with clear messages; learn advanced features later

R Markdown Overview

Reproducible reports combining code, text, and output:

```
# Create new R Markdown: File > New File > R Markdown

# Basic structure in .Rmd file:
# ---
# title: "My Analysis"
# author: "Your Name"
# date: "2025-02-08"
# output: html_document
# ---

# Code chunks start with:
# ```{r}
# gss <- read_csv("data/GSSsubset.csv")
# glimpse(gss)
# ```

# Inline code: The mean income is `r mean(gss$income, na.rm = TRUE)`.

```

Project Structure Guidelines

Recommended folder organization:

- `data/raw/` - Original, immutable data
- `data/processed/` - Cleaned, transformed data
- `scripts/` - Numbered scripts: 01_import.R, 02_clean.R
- `results/figures/` - Saved plots
- `results/tables/` - Exported tables
- `docs/` - Documentation and notes
- `.gitignore` - Git exclusions
- `README.md` - Project overview

Key Principles:

- Raw data is never modified
- Scripts are numbered for execution order
- Results are separated from code
- README explains how to reproduce analysis

Tip: Start with this structure; adapt as project grows

Performance Considerations

Write efficient R code:

```r
# 1. Vectorize operations (avoid loops when possible)
# Slow:
result <- c()
for (i in 1:1000) { result[i] <- i^2 }
# Fast:
result <- (1:1000)^2

# 2. Pre-allocate vectors
# Slow: x <- c(); for (i in 1:1000) { x <- c(x, i) }
# Fast: x <- numeric(1000); for (i in 1:1000) { x[i] <- i }

# 3. Use data.table for large datasets
# library(data.table)
# dt <- fread("large_file.csv")  # Faster than read.csv

# 4. Profile code to find bottlenecks
# install.packages("profvis")
# profvis::profvis({ your code here })
```

Resources for Learning

Books:

- Wickham and Grolemund (2017). R for Data Science
- Wickham (2015). R Packages
- Peng (2015). R Programming for Data Science

Online Courses:

- DataCamp: R track
- Coursera: R Programming (Johns Hopkins)
- edX: Data Science with R (Microsoft)

Reference Sites:

- R Documentation: cran.r-project.org/manuals.html
- RStudio Cheat Sheets: rstudio.com/resources/cheatsheets
- Stack Overflow: stackoverflow.com/questions/tagged/r

Practice:

- Reproduce analyses from papers or blogs

Practice Exercises

Try these with GSSsubset:

1. Import the data and check its dimensions
2. Calculate mean income by gender
3. Create a new variable: income per year of age
4. Filter to respondents over 40 with income greater than 30000
5. Make a histogram of age with 15 bins
6. Count missing values in each column
7. Convert degree variable to ordered factor
8. Save your cleaned dataset as CSV

Challenge:

- Write a function that summarizes any numeric variable
- Create an R Markdown report with your findings
- Set up a Git repository for your work

Tip: Start with the basics, then add complexity

Next Steps

After mastering basics:

1. Learn tidyverse verbs: filter, select, mutate, summarize
2. Master ggplot2 for publication-quality graphics
3. Explore statistical modeling: lm, glm, lmer
4. Learn data wrangling with tidyr and stringr
5. Practice reproducible reporting with R Markdown
6. Contribute to open source or your own packages

Session 2 Preview:

- Data import, cleaning, and exploratory analysis
- Working with missing data and outliers
- Univariate and bivariate exploration
- Introduction to dplyr for data manipulation

Remember: Consistent practice is more important than speed

Summary

Key Takeaways:

1. R is powerful, free, and designed for data science
2. RStudio provides an efficient workflow environment
3. Understand data types: vectors, lists, data frames, factors
4. Always inspect data after import
5. Handle missing values thoughtfully
6. Use scripts and projects for reproducibility
7. Learn tidyverse for modern data science workflow
8. Get help using ?function and online resources

Golden Rules:

- Test code in small chunks
- Comment your reasoning, not just your code
- Keep raw data immutable
- Make your work reproducible for others

You now have the foundation to start your data science journey in R

Questions?

Common Topics:

- Installing packages and troubleshooting
- Understanding error messages
- Choosing between base R and tidyverse
- Setting up RStudio for efficiency
- Next learning priorities

No question is too basic - we all started somewhere

Thank You!

See you in Session 2

Contact: cdam@chuka.ac.ke
Resources: https://cdam.chuka.ac.ke
Code: https://github.com/cdam-chuka/r-sessions

Session 2: Data Import, Cleaning, and EDA
Preparation: Install tidyverse, review today's code